



3. Formularios y controles

3.1. Interacción con formularios

Los formularios de una página web son accesibles a través de la colección `document.forms`. Se puede hacer referencia a través del índice o del nombre del formulario (atributo `name`). Dentro del formulario, los elementos de este son accesibles mediante la colección `form.elements`, también a través de sus propiedades `name`. Cada elemento tiene acceso a los datos que almacena mediante la propiedad `value`. Por ejemplo, dados los siguientes formularios:

```
<form name="datospersonales">
  <input name="nombre" type="text" value="José">
  <input name="apellido" type="text" value="Pérez">
</form>

<form name="datoslaborales">
  <input name="profesion" type="text" value="desarrollador web">
  <div class="detalles">
    <input name="cargo" type="text" value="programador junior">
  </div>
</form>
```

es posible hacer referencia a los valores que almacenan los campos de la siguiente manera:

```
let formDatosPersonales = document.forms.datospersonales;
// o también mediante document.forms[0]

let formDatosLaborales = document.forms.datoslaborales;
// o también mediante document.forms[1]

let nombre = formDatosPersonales.elements.nombre.value; // José
let apellidos = formDatosPersonales.elements.apellidos.value; // Pérez

let profesion = formDatosLaborales.elements.profesion.value; // desarrollador
web
// Da igual que <input name="cargo"> esté dentro de un 'div' para acceder a
su propiedad
let cargo = formDatosLaborales.elements.cargo.value; //programador junior
```

A su vez, los elementos pueden acceder al elemento formulario a través de su propiedad `form`.

```
let formDatosPersonales = document.forms.datospersonales;

let elemNombre = formDatosPersonales.elements.nombre;

// Acceso al formulario desde el campo
console.log(elemNombre.form); // Elemento <form name="datospersonales">
```

Los elementos de tipo `checkbox` o `radio` pueden agruparse mediante el mismo atributo `name`. En esa situación, si se accede a ellos a través de `elements` se devolverá una colección. En el caso de los elementos `radio`, es posible seguir accediendo a la propiedad `value` de dicha colección para obtener su valor; en el caso de los elementos `checkbox`, no (habrá que recorrer el `array` y obtener el valor de cada elemento). Además, esos elementos definen la propiedad `checked`, que devuelve un booleano que indica si está o no

activado. Consideremos el siguiente formulario:

```
<form name="encuesta">
  <div>
    Indica tu edad:
    <input name="edad" type="radio" value="menor20">
    <input name="edad" type="radio" value="mayor20" checked>
  </div>
  <div>
    Selecciona tus opciones de suscripción:
    <input name="suscripcion" type="checkbox" value="javascript" checked>
    <input name="suscripcion" type="checkbox" value="java">
    <input name="suscripcion" type="checkbox" value="go" checked>
  </div>
</form>
```

A continuación, veamos cómo acceder a los campos y sus valores:

```
let formEncuesta = document.forms.encuesta;

let elemsEdad = formEncuesta.elements.edad;
console.log(elemsEdad[0]); // Elemento <input name="edad" type="radio"
value="menor20">
console.log(elemsEdad[1]); // Elemento <input name="edad" type="radio"
value="mayor20" checked>

console.log(elemsEdad.value); // mayor20
console.log(elemsEdad[0].checked); // false
console.log(elemsEdad[1].checked); // true

// Activamos la primera opción desde JavaScript
elemsEdad[0].checked = true;
// Comprobamos los cambios
console.log(elemsEdad.value); // menor20
console.log(elemsEdad[0].checked); // true
console.log(elemsEdad[1].checked); // false

let elemsSuscripcion = formEncuesta.elements.suscripcion;
console.log(elemsSuscripcion[0]); // Elemento <input name="suscripcion"
type="checkbox" value="javascript" checked>
console.log(elemsSuscripcion[1]); // Elemento <input name="suscripcion"
type="checkbox" value="java">
console.log(elemsSuscripcion[2]); // Elemento <input name="suscripcion"
type="checkbox" value="go" checked>

console.log(elemsSuscripcion.value); // "" (cadena vacía)
console.log(elemsSuscripcion[0].checked); // true
console.log(elemsSuscripcion[1].checked); // false
console.log(elemsSuscripcion[2].checked); // true
```

Por último, los elementos `select` tienen acceso a las propiedades `value`, `options` y `selectedIndex`. Los elementos `option` tienen acceso a la propiedad `selected`. A continuación, se muestra un ejemplo:

```
<select id="lenguaje">
  <option value="javascript">JavaScript</option>
  <option value="java">Java</option>
  <option value="go">Go</option>
</select>

<script>
  let lenguajeSelect = document.getElementById("lenguaje");
  lenguajeSelect.options[2].selected = true; // Opción 2
  console.log(lenguajeSelect.value); // go
  console.log(lenguajeSelect.selectedIndex); // 2

  lenguajeSelect.selectedIndex = 1; // Opción 1
  console.log(lenguajeSelect.value); // java
  console.log(lenguajeSelect.selectedIndex); // 1

  lenguajeSelect.value = "javascript"; // Opción 0
  console.log(lenguajeSelect.value); // javascript
  console.log(lenguajeSelect.selectedIndex); // 0
</script>
```

! IMPORTANTE

En la mayoría de los casos, es imprescindible definir los atributos `id` y `name` al crear campos de formulario.

El atributo `id` se utiliza para crear las etiquetas asociadas a los campos: el atributo `for` del elemento `<label>` debe corresponderse con el atributo `id` del campo de formulario asociado. Con el fin de simplificar el código, los ejemplos no incluyen las etiquetas, pero es necesario recordar que es obligatorio utilizarlas desde el punto de vista de la corrección del documento HTML y, por supuesto, de la accesibilidad.

El atributo `name` es útil para acceder a los campos de formulario desde JavaScript, tal como hemos visto, pero su función principal es la de enviar los datos al servidor. Los campos de formulario que no incluyan el atributo `name` no se envían mediante el método convencional de envío de formularios. [Aclaración: pueden enviarse si en la lógica de cliente se accede a sus datos y se envían por medio de una petición AJAX, pero en este caso no estaríamos utilizando el método convencional de envío de formularios].

3.2. Eventos de formularios

Los eventos más utilizados con los campos de formulario son:

- **focus**. Se activa cuando un campo obtiene el foco.
- **blur**. Se activa cuando un campo pierde el foco.
- **change**. Se activa cuando el campo ha sido modificado, al perder el foco.
- **input**. Se activa cuando el campo se modifica [inmediatamente].
- **submit**. Se activa cuando se hace clic en un botón de envío o se pulsa la tecla **Enter** en un campo de texto de formulario. Veremos este evento con más detalle en la siguiente sección.

EJEMPLO

En la documentación complementaria de la unidad, situada en la sección *Material de trabajo / Descarga de documentos*, dispones del archivo `DWEC_U03_A03_01.html`, con el código de ejemplo.

3.3. Envío y validación de formularios

En primer lugar, es necesario distinguir entre el envío de un formulario, el evento **submit** y la función **submit()** que se puede ejecutar sobre un elemento de tipo formulario.

El envío de un formulario consiste en **enviar la información de dicho formulario** a una URL de servidor. A no ser que se indique lo contrario, cuando hablemos de «envío de formulario» nos referiremos al **método convencional de envío**, no al envío de datos mediante funciones personalizadas a través de AJAX.

El **envío** de un formulario se puede realizar de **dos maneras**:

- Cuando se produce un **evento** de tipo **submit** en el formulario. Como hemos visto en secciones anteriores, este evento se produce al hacer clic en un botón de tipo **submit** o al pulsar **Enter** sobre un campo.
- Utilizando la función **form.submit()** desde JavaScript.

! IMPORTANTE

La ejecución de la función **submit()** únicamente envía el formulario: **no provoca la generación del evento submit**.

En ocasiones, hay cierta confusión sobre cómo proceder a la hora de crear una **función personalizada de envío**. Por ejemplo, es posible que se desee realizar una **validación de datos** antes de proceder al envío; o, en ocasiones, no se quiere realizar un envío convencional y en su lugar se desea **obtener los datos de los campos** y enviarlos por otro método a través de una petición **AJAX**.

La recomendación general es utilizar el evento **submit** del formulario, en lugar de utilizar el evento **click** de un botón de envío. Las ventajas de ello son:

- Se controla tanto el envío a través del botón como a través de la tecla **Enter**.
- Si se utiliza **validación HTML**, esta se realiza cuando se produce el evento **submit**, antes de la ejecución del código de la función manejadora de dicho evento.

! IMPORTANTE

Si no se desea utilizar la validación HTML, es posible hacerlo utilizando el atributo **novalidate** en el formulario.

Sin embargo, como hemos visto, el evento **submit** provoca por defecto el envío del formulario: si queremos, por ejemplo, realizar una comprobación adicional para decidir si se envían o no los datos, será necesario impedir dicho comportamiento por defecto. Para ello puede utilizarse **event.preventDefault()**.

EJEMPLO

En la documentación complementaria de la unidad, situada en la sección *Material de trabajo / Descarga de documentos*, dispones del archivo **DWEC_U03_A03_02.html**, con el código de ejemplo.

Para terminar, desde JavaScript es posible interactuar también con el **mecanismo de validación de HTML5** mediante las funciones **form.checkValidity()** y **form.reportValidity()**. La diferencia entre ambas es que **checkValidity** no muestra los mensajes de validación estándar del navegador, y **reportValidity**, sí.

EJEMPLO

En la documentación complementaria de la unidad, situada en la sección *Material de trabajo / Descarga de documentos*, dispones del archivo **DWEC_U03_A03_03.html**, con el código de ejemplo.